E-tiGo Developer Handoff v2: Finance, Scheduling, EV & Maintenance, Third-Party Booking, Disputes

Oct 22, 2026 · For Designer, Frontend and Mobile teams · Replaces the 20 October draft

How to read this document

All five modules are built and running on staging. This version was checked against the backend code on 21 October 2026, not only the Postman overview. Every open question in the 20 October draft now has an answer.

Each rule carries one of these tags:

Tag	Meaning	Can you build against it?
Built	In the code on staging, with tests	Yes
Built (default)	In the code, but the value was picked by the backend team and the client has not confirmed it. The value may change; the field and endpoint will not	Yes. Read the value from the API, do not hard-code it
Client decision	Confirmed by the client	Yes
Not built	No backend support yet	No. Do not show the feature
Proposal	A design idea, not backend behaviour	Design only

The detailed per-feature guides live in the API repo under `doc/technical/`. This handoff summarises them and links each module to its guide. The full endpoint list is in `doc/technical/API_REFERENCE.md` and the Postman collection *E-tiGo API v1*.

What changed since the 20 October draft

Area	20 Oct draft said	Correct state
Rider-scheduled rides (2B)	No evidence, may be unbuilt	Built (PRs #91, #94, #95). Guide: `FRONTEND_UPDATE_SCHEDULED_RIDES.md`

Area	20 Oct draft said	Correct state
Third-party booking	No evidence found	Built (PR #104). Booker chooses who gets the PIN and tracking (PR #120). Guides: `FRONTEND_UPDATE_THIRD_PARTY_BOOKINGS.md`, `MOBILE_UPDATE_THIRD_PARTY_SHARE_WITH.md`
Disputes	States and evidence rules not visible	Built (PR #106) with full status list, categories, evidence limits and SLAs
Lost & Found	Statuses and handover unknown	Built (PR #105)
Driver scheduling and maintenance	Paths unknown	Built (PR #110), with repair attachments added after
EV stations	Paths unknown	Built, admin shape unified (PRs #116, #117)
Endpoint paths and fields	Not known	Real paths are listed in every module below
Status tags	Confirmed, Needs backend confirmation, Proposal	Replaced by the word tags above

Two backend gaps found during this review were fixed on 22 October in PR #121: Operations can now review EV charge requests (section 3), and drivers still see a paused fleet agreement (section 3, Fleet agreement pause and swap).

Global conventions

These rules apply to every module below. All are **Built**.

Topic	Rule
Base URL	`{APP_URL}/api/v1`
Auth (apps)	`POST /auth/otp/send` → `POST /auth/otp/verify` (with `type`: `passenger` or `driver`) → `POST /auth/register/complete` if new. Then `Authorization: Bearer {token}`
Auth (admin)	`POST /admin/auth/login` with email and password

Topic	Rule
Dual accounts	One phone can hold a passenger account and a driver account. Each has its own token
Token life	Admin 8 hours, apps 30 days. Expired token → `401 UNAUTHENTICATED`
Rate limits	Auth routes: 5 per minute per IP. Many other routes have their own limits (payout request 3/min, top-up 10/min, dispute evidence 10/min). All return `429` with `Retry-After` in seconds
IDs	UUIDs. A malformed id in a filter returns `422 INVALID_PARAMETER`
Pagination	Lists take `per_page` and `page` and return a `meta` block: `current_page`, `last_page`, `per_page`, `total`. `per_page` is clamped to each endpoint's range
Content type	JSON, or `multipart/form-data` for uploads

Money formats (corrected)

The 20 October draft said all money arrives as decimal strings. That is only true for some fields. The units differ by area:

Area	Format	Example for ₦5,000
Wallet balance, wallet transactions, ledger, payouts, adjustments, EV kWh price	Integer **kobo**	`500000`
Ride fares (`fare_estimate_amount`, `final_fare_amount`, `discount_amount`), fleet remittance	Decimal string in **naira**	`"5000.00"`
Ride `payment.amount`, `payment.tip_amount`	Number in naira	`5000`
Request bodies	Follow the field's own unit: payout `amount` is kobo, tip `amount` is naira	

The client must not do money maths. Divide kobo by 100 only to display it. Read the unit from the API reference for every new field.

Errors

Every error returns `{ "message": "...", "error_code": "..." }`. Validation errors add `errors`, keyed by field.

Status	`error_code`	When	UI handling
401	`UNAUTHENTICATED`	Missing or expired token	Send to sign-in
403	`HTTP_ERROR`	Wrong user type or admin role	Hide the action. Show "You don't have access"
404	`NOT_FOUND`	Record missing, or not yours	Empty or not-found state
409	`INVALID_STATE_TRANSITION`	A **ride or SOS** changed state under the request. Body has `current_status`	Refresh, then show the new state
409	`HTTP_ERROR`	Other conflicts, for example a payout blocked by its bank account (`bank_account_block`)	Show the `message` , refresh the row
422	`VALIDATION_ERROR`	Bad input, or an action the record's state does not allow (most finance, dispute and maintenance actions)	Show `errors` inline, or the `message` as a toast. A `current_status` in the body means the row is stale: refresh it
422	`INVALID_PARAMETER`	Malformed id or date	Log it, show a generic error
429	`HTTP_ERROR`	Rate limit	Countdown from `Retry-After`
502	`HTTP_ERROR`	Paystack could not be reached. Nothing changed	Keep the button enabled, allow retry
500	`SERVER_ERROR`	Server fault. Staging adds a `details` block	Generic retry state

Correction: the draft said payout and refund actions return 409. Most return 422. Only ride and SOS state changes use `409 INVALID_STATE_TRANSITION` .

Two more codes to handle: `422` with `error_code: BALANCE_DUE` and a `balance_due` amount from `POST /rides` when the passenger owes money (show the Pay balance screen), and some business-rule 422s that carry only a `message` with no `error_code` .

Always fall back to showing `message`.

Admin roles

There are six admin roles. `super_admin` passes every role check. The table lists only the access that matters for these five modules, taken from the route guards. All rows are **Built**.

Role	Value	Access in these modules
Super Admin	`super_admin`	Everything. Also tier 3 disputes, dispute settings, admin invitations
Operations	`operations`	Driver shifts, availability coverage, vehicle maintenance, EV stations, scheduled-ride assignment and settings, third-party and Lost & Found settings, offline flags, Lost & Found queue, fleet vehicles and agreements, operational reports. **Wallets read-only** (added in PR #103; the draft said none)
Safety Operator	`safety_operator`	SOS console, offline flags, the Safety dispute queue (sees every dispute), dispute refund requests, passenger and driver records
Support	`support`	Tier 1 disputes, dispute refund requests, Lost & Found queue, passenger records. Read-only: wallets, driver ledgers, payout accounts, balances due
Support Lead	`support_lead`	Everything Support has, plus tier 2 disputes (escalations, appeals, missed deadlines) and larger refund approvals
Finance	`finance`	Wallets including freeze and unfreeze, ledger, adjustments, payouts, payout-account review, ride refunds, balance-due waivers, commission, wallet and tip settings, all reports, fleet agreements, pricing, promos, and the processing of dispute refund requests

Three notes for the dashboard:

- Hide, don't just disable, any action a role cannot take. The API returns `403` either way.

- Fleet shift mode (`PUT /admin/fleet-vehicles/{id}/shift-mode`) is **Operations only**. The draft listed Finance too.
- No seeded Support Lead account exists. A super admin can invite one from `POST /admin/admins/invite` with role `support_lead` .

Module status at a glance

Every module is built and on staging. What is still open are product values the client has not confirmed, not missing endpoints.

Module	Backend status	Main PRs	Frontend guide in `doc/technical/`
Finance	Built	#75, #81, #84–#87, #101, #103	`FRONTEND_UPDATE_PAYMENTS_EV_SURGE.md` , `MOBILE_UPDATE_ADD_CARD_WALLET_CREDIT.md`
Scheduling 2A: driver shifts and availability	Built	#110	`FRONTEND_UPDATE_DRIVER_SCHEDULING_MAINTENANCE.md`
Scheduling 2B: rider-scheduled rides	Built	#91, #94, #95	`FRONTEND_UPDATE_SCHEDULED_RIDES.md`
EV charging	Built	#75, #81, #116, #117, #121	`FRONTEND_UPDATE_PAYMENTS_EV_SURGE.md` , `FRONTEND_UPDATE_ADMIN_DESIGN_GAPS.md`
Vehicle maintenance	Built, plus repair attachments	#110 and later commits	`FRONTEND_UPDATE_DRIVER_SCHEDULING_MAINTENANCE.md`
Third-party booking	Built	#104, #120	`FRONTEND_UPDATE_THIRD_PARTY_BOOKINGS.md` , `MOBILE_UPDATE_THIRD_PARTY_SHARE_WITH.md`
Advanced disputes	Built	#106	`FRONTEND_UPDATE_ADVANCED_DISPUTES.md`
Offline trip flags	Built	#114, #115	`MOBILE_UPDATE_OFFLINE_FLAGS.md` , `FRONTEND_UPDATE_OFFLINE_FLAGS_DESIGN_GAPS.md`
Lost & Found	Built	#105	`FRONTEND_UPDATE_LOST_AND_FOUND.md`

1. Finance

Finance is **Built** for all three apps. Passengers hold a wallet. Drivers hold an earnings account and withdraw to a verified bank account. Finance staff run wallets, ledger, adjustments, payouts, refunds and reports. Guide: `FRONTEND_UPDATE_PAYMENTS_EV_SURGE.md`.

Where a fare goes

Driver type	Money path	Ledger lines the driver sees
Regular	Fare → driver earnings, minus platform commission (default **20%**, set by Finance in `/admin/settings/commission`)	Card or wallet ride: the full fare as a credit (`ride_fare`) and the commission as a separate debit (`commission`). Cash ride: only the commission debit
Fleet (hire-purchase)	Fare → 100% to E-tiGo until the daily target is met → then 100% to the driver	No commission line, ever

Worked fleet example, target ₦40,000 (unchanged from the Fleet handoff):

Ride	Fare	To E-tiGo	To driver	Remitted so far
9	₦6,000	₦6,000	₦0	₦38,000
10	₦5,500	₦2,000	₦3,500	₦40,000, target met
11	₦4,200	₦0	₦4,200	n/a

Endpoints: passenger app

Action	Endpoint	Notes
Wallet balance	`GET /passenger/wallet`	`balance`, `available`, `held` in kobo. `status`: `active`, `frozen`, `closed`
Wallet history	`GET /passenger/wallet/transactions`	Filters `type` (`debit`, `credit`), `from`, `to`. 20 per page

Action	Endpoint	Notes
Top up	`POST /passenger/wallet/topup`	`amount` in kobo, minimum 50000 (₦500). Returns `payment_link` and `tx_ref`
Confirm top-up	`GET /passenger/wallet/topup/{tx_ref}/verify`	Safe to call twice. The Paystack webhook credits it anyway
Add a card	`POST /payments/initialize`, then `GET /payments/{id}/verify`	The ₦100 card check is credited to the wallet as a top-up (PR #101)
Pay a ride	`payment_method` on `POST /rides`	`cash`, `card`, `wallet`, `bank_transfer`
Tip	`POST /rides/{ride}/tip`	`amount` in naira within `GET /rides/tip-settings`. Bank-transfer rides must send `source`: `wallet` or `card`
Balance due	`GET /passenger/balance-due`, `POST /passenger/balance-due/pay`	Unpaid fees (for example a scheduled-ride late-cancellation fee). Blocks new bookings by default

Endpoints: driver app

Action	Endpoint	Notes
Balances	`GET /driver/ledger/summary`	`pending`, `available`, `total_paid` in kobo
Earnings history	`GET /driver/ledger/transactions`, `GET /driver/ledger/transactions/{entry}`	Each entry has `transaction_type` (value and label)
Earnings overview	`GET /driver/earnings`, `GET /driver/earnings/rides/{ride}`	Per-ride breakdown
Bank account	`GET/POST /driver/bank-account`, `GET /driver/bank-account/banks`, `POST /driver/bank-account/resolve`, `POST /driver/bank-account/otp`	Every save needs an OTP, including the first account. Name check and a 24-hour hold apply

Action	Endpoint	Notes
Withdraw	`POST /driver/payouts`	`amount` in kobo, minimum 100000 (₦1,000). 3 per minute
Payout history	`GET /driver/payouts`	Status per payout
Payout preference	`PUT /driver/payouts/preference`	`on_demand` or `weekly_auto` (Mondays 06:00 Lagos)
Fleet remittance	`GET /driver/remittance/today`, `GET /driver/remittance/history`, `GET /driver/fleet-agreement`	Fleet drivers only

Endpoints: admin dashboard

Area	Endpoints	Roles
Wallets	`GET /admin/wallets`, `/{account}`, `/{account}/transactions`, `/{account}/audit-trail`	Finance, Support, Support Lead, Operations (read)
Freeze	`POST /admin/wallets/{account}/freeze`, `/unfreeze`	Finance
Driver ledgers	`GET /admin/driver-ledgers`, `/{account}`	Finance, Support, Support Lead (read)
Ledger explorer	`GET /admin/ledger`, `GET /admin/ledger/export`	Finance
Adjustments	`GET/POST /admin/adjustments`, `POST /{id}/approve`, `/reject`	Finance. A second Finance admin must approve: the creator cannot approve their own
Payouts	`GET /admin/payouts`, `POST /{id}/approve`, `/reject`, `/retry`, `/verify`, `GET /admin/payouts/export`	Finance
Payout accounts	`GET /admin/payout-accounts`, `/{id}`; `POST /{id}/approve`, `/reject`, `/release-hold`	Read: Finance, Support, Support Lead. Actions: Finance

Area	Endpoints	Roles
Ride refund	`POST /admin/rides/{ride}/refund`	Finance. Refunds to the wallet
Balances due	`GET /admin/balances-due`, `POST /{id}/waive`	Read: Finance, Support, Support Lead. Waive: Finance
Settings	`/admin/settings/commission`, `/wallet`, `/tips`	Finance
Reconciliation	`GET /admin/reports/reconciliation`, `/history`, `/wallet-liability`	Finance
Reports	`GET /admin/reports/ride-volume`, `/completion-rate`, `/revenue`, `/driver-utilisation`, `/export`	Operations, Finance

Exports are **CSV only**. There is no PDF export.

Enums

Enum	Values
Wallet status	`active`, `frozen`, `closed`
Ledger entry direction	`debit`, `credit`
Transaction type (`transaction_type.value`)	`top_up`, `ride_payment`, `ride_settlement`, `commission`, `tip`, `refund`, `cancellation_fee`, `balance_due`, `cash_change`, `ev_reservation_fee`, `fleet_remittance`, `adjustment`, `reversal`, `payout`
Top-up status	`pending`, `completed`, `abandoned` (after 30 min), `failed`
Payout status	`requested`, `approved`, `processing`, `paid`, `failed`, `reversed`, `rejected`
Adjustment	type `credit` or `debit`; status `pending`, `approved`, `rejected`
Balance due	`outstanding`, `paid`, `waived`

Enum	Values
Ride payment status	`pending`, `authorized`, `captured`, `settled`, `refunded`, `failed`, `pending_collection`, `collected`, `paid`

Payout lifecycle: `requested` → `approved` → `processing` → `paid`. Exits: `rejected` (Finance declined, nothing left the balance), `failed` then `reversed` (money back in the balance, Retry allowed). Show `approved` and `processing` as in progress however long they take.

Wallet limits (Built (default), Finance can change them)

Setting	Default
Minimum top-up	₦500
Maximum wallet balance	₦500,000
Daily top-up cap	₦100,000; at most 5 top-ups an hour, 10 a day
Minimum payout	₦1,000
Driver balance may go negative to	−₦5,000
Hold after a bank-account change	24 hours

Visibility (corrected)

Data	Passenger	Regular driver	Fleet driver	Support	Finance
Own wallet or earnings balance	Yes	Yes	Yes	n/a	n/a
Any wallet	No	No	No	Read-only	Full
Commission line	n/a	Yes	Never	Read-only	Yes
Remittance and personal earnings	n/a	n/a	Yes	No	Yes
`total_vehicle_cost`, `remaining_amount`	No	No	**Yes**	No	Yes

Correction: the draft said fleet drivers never see the vehicle price. The client reversed that decision. `GET /driver/fleet-agreement` returns `total_vehicle_cost`, `remaining_amount`, `total_remitted` and `progress_percentage` (**Client decision**, recorded in `doc/client/FLEET_MANAGEMENT_COMPLETE.md`). Shortfall flags and settlement amounts stay admin-only.

Business rules for the UI

1. Never do money maths in the client. Divide kobo by 100 for display only.
2. Hide, don't disable, freeze, adjustment, payout and refund actions for any role but Finance.
3. A frozen wallet blocks top-ups, wallet payments, wallet tips and adding a card. Show a support contact.
4. When `payout_eligibility.can_request_payouts` is false, disable Withdraw and show its `message` (`payout_account_on_hold`, `payout_account_under_review`, `payout_account_rejected`).
5. Label each history row from `transaction_type.label`. Refunds, tips, adjustments, remittance and payouts are separate types, so show them as their own rows.
6. Don't add a "mark as paid" control. Use Check status (`/verify`).
7. Fleet remittance `settled` flips daily at **04:00 Lagos** for the day before (Built). Before that, show today's figure as "so far".

Notifications

Push and in-app: `topup_success`, `topup_failed`, `ride_wallet_payment`, `wallet_refund`, `payout_approved`, `payout_paid`, `payout_failed`, `payout_rejected`, `payout_account_changed`, `payout_account_reviewed`, `balance_due`. There is **no** notification when a wallet is frozen (Not built). Wallet and ledger balances are not pushed over the websocket. Refresh on screen open and when one of these notifications arrives.

2. Scheduling

Both meanings of scheduling are **Built**: 2A is driver shifts and availability, 2B is passengers booking a ride for later. The draft's "2B not found" was wrong. All times in the API are UTC in ISO 8601. Show them in the device's local time (Lagos for all live cities).

2A. Driver shifts and availability

Guide: `FRONTEND_UPDATE_DRIVER_SCHEDULING_MAINTENANCE.md`. Audience: driver app and admin dashboard (Operations).

Two separate things were built:

- **Availability plans**, for every driver. A driver says when and where they expect to drive. Plans are optional, never block going online, and passengers never see them.
- **Shifts**, only for E-tiGo fleet cars in **shift mode**, which two or more drivers share. A shift controls when the driver can go online in that car.

Action	Endpoint	Who
My shifts	`GET /driver/shifts` (`include_past=1` adds 30 days)	Driver. Show the screen only when `shift_driver` is true
My availability	`GET/POST /driver/availability-plans`, `DELETE /driver/availability-plans/{id}`	Driver. 30 min to 12 h each, up to 14 days ahead, `repeat_weekly` 0–3
Shift mode on a car	`PUT /admin/fleet-vehicles/{id}/shift-mode`	Operations
Timetable	`GET /admin/driver-shifts` (filters `vehicle_id`, `driver_id`, `status`, `from`, `to`; `summary` per status)	Operations
Assign or build a rota	`POST /admin/driver-shifts` with `repeat_daily_for` 0–13	Operations
Edit, cancel	`PUT /admin/driver-shifts/{id}`, `POST /admin/driver-shifts/{id}/cancel`	Operations
Coverage	`GET /admin/availability-coverage?city_id&from&to` (up to 14 days)	Operations
Settings	`GET/PUT /admin/settings/driver-scheduling`	Operations

Shift statuses: `scheduled`, `active`, `completed`, `missed`, `cancelled`.

Shift rules (Built (default), Operations settings):

1. Go online opens 15 minutes before the shift (`online_from`). New offers stop 15 minutes before it ends (`offers_stop_at`).
2. Not online 30 minutes after the start → `missed`, and Operations get an email.
3. Overlaps on the car or the driver return 422 naming the clash. Overnight shifts work.
4. Once a shift is in progress, only `ends_at` can change. Only shifts not yet started can be cancelled.
5. Suspending or sanctioning a driver ends their current shift and cancels upcoming ones.
6. A car on a hire-purchase agreement cannot use shift mode. So shifts never affect a fleet daily target or shortfall streak.

Go online (`POST /driver/toggle-online`) can now return 422 with `reasons` for shift-window problems and for a car whose papers expired. Show every reason as given. A 200 adds `vehicle_notice` and `shift` .

Notifications: `driver_shift_assigned` , `driver_shift_updated` , `driver_shift_cancelled` , `driver_shift_missed` , `driver_shift_ending` , `driver_shift_ended` , `shift_handover_delayed` .

2B. Rider-scheduled rides

Guide: `FRONTEND_UPDATE_SCHEDULED_RIDES.md` . Audience: all three apps. Client decisions are in `doc/client/SCHEDULED_RIDES_DECISIONS.md` .

Breaking: ride `status` has a new value, `scheduled` . Add it to every exhaustive `switch` or enum decoder. A scheduled ride is not active, has no PIN yet, and never appears in `GET /passenger/active-ride` .

Action	Endpoint	Notes
Read the limits	`GET /config` → `scheduled_rides`	Hide Schedule when `enabled` is false
Quote	`POST /rides/estimate` with `scheduled_pickup_at`	Priced for the pickup time
Book	`POST /rides` with `scheduled_pickup_at`	ISO 8601 **with a UTC offset**, or 422. Returns `status: scheduled` , `pin_code: null`
Upcoming list	`GET /rides/scheduled`	Soonest first
Edit	`PUT /rides/{ride}/scheduled`	Only while `scheduled.can_edit` . A new time or pickup releases the reserved driver
Cancel	`DELETE /rides/{ride}/scheduled`	Show `scheduled.cancellation_fee_if_cancelled_now` first
Driver: browse, reserve, release	`GET /driver/scheduled-rides/available` , `POST /driver/scheduled-rides/{ride}/reserve` , `GET /driver/scheduled-rides` , `POST /driver/scheduled-rides/{ride}/release`	Up to 5 reservations, 60 min apart
Admin: list, assign, settings	`GET /admin/scheduled-rides` , `POST/DELETE /admin/scheduled-rides/{ride}/assign` , `GET/PUT /admin/settings/scheduled-rides`	Operations

The draft's open questions, answered:

Topic	Answer	Tag
Lead time	30 minutes to 3 days ahead	Client decision
Bookings per passenger	3 upcoming	Client decision
Time zone	`scheduled_pickup_at` is stored and returned in UTC. Send it with an offset	Built
Edit window	Until `dispatch_at`, 15 minutes before pickup	Built (default)
Cancel fee	None for reservations. Late cancellation (under 30 min, after a driver reserved) costs the city's Economy base fare, paid to that driver	Client decision
Unpaid fee	Becomes a **balance due**. `POST /rides` then returns 422 `error_code: BALANCE_DUE` until paid	Client decision
Driver visibility	Drivers can see and reserve bookings in their city and class as soon as they are made	Built (default)
At dispatch	A reserved, online driver gets the ride directly. Otherwise it goes to normal matching	Built (default)
No driver found	Normal no-driver handling. The system cancels with a `dispatch_failure_reason` if it cannot start: `insufficient_wallet_balance`, `wallet_unavailable`, `account_suspended`, `service_unavailable`, `missed_dispatch_window`, `city_paused`	Built
Fare	Quoted for the pickup time. Time-based and manual surge at that time apply; live demand surge never does	Built (default)
Recurring bookings	Not supported	Not built
For someone else	Yes, a scheduled ride can be a third-party booking	Built

The `scheduled` object on a ride carries `scheduled_pickup_at`, `dispatch_at`, `can_edit`, `reserved_driver`, `reservation_fee`, `cancellation_fee_if_cancelled_now`, `late_cancellation_fee`, `late_cancellation_fee_status` and

`dispatch_failure_reason`.

The draft's proposed states (`driver_assigned`, `reminder_sent`, `expired`) do not exist. A scheduled ride uses the normal ride statuses: `scheduled` → `searching` or `driver_en_route` → the usual trip statuses, or `cancelled`.

Notifications: `scheduled_ride_reminder` (60 min before), `scheduled_ride_reserved`, `scheduled_ride_reservation_released`, `scheduled_ride_dispatched`, `ride_cancelled`, `balance_due`.

3. EV charging and maintenance

Stall reservations, station management, maintenance tracking, repair requests and fleet pause are **Built**. Two gaps found while writing this handoff were fixed on 22 October (PR #121): Operations can now review EV charge requests, and drivers still see a paused fleet agreement. Guides: `FRONTEND_UPDATE_PAYMENTS_EV_SURGE.md` (EV) and `FRONTEND_UPDATE_DRIVER_SCHEDULING_MAINTENANCE.md` (maintenance).

Fuel types (corrected)

The draft listed four fuel types. There are six: `petrol`, `diesel`, `cng`, `hybrid`, `plug_in_hybrid`, `electric`. `fuel_type` is required when registering a vehicle. Every vehicle response has `can_charge`, which is true for `electric` and `plug_in_hybrid` only. Show EV screens only when `vehicle.can_charge` is true. A plain `hybrid` cannot charge.

EV charging

Action	Endpoint	Who
Stations in a city	`GET /ev-stations?city_id=`, `GET /ev-stations/{station}`	Anyone, no login. Sort by distance in the app
Reserve a stall	`POST /driver/ev-reservations/stations/{station}/reserve`	Driver with `can_charge`
My reservations	`GET /driver/ev-reservations`, `GET /driver/ev-reservations/{id}`	Driver
Start, finish, cancel	`POST /driver/ev-reservations/{id}/activate`, `/complete`, `/cancel`	Driver

Action	Endpoint	Who
Manage stations	`GET/POST /admin/ev-stations`, `GET/PUT/DELETE /admin/ev-stations/{id}`, `PATCH /{id}/status`, `PUT /{id}/stalls`, `GET /admin/ev-stations/utilisation`	Operations

A reserve call has three outcomes: `reserved` (201, stall held), `queued` (200, a stall frees up soon) or `retry` (200, nothing free).

Reservation lifecycle: `reserved` → `active` (driver taps Start) → `completed`. Exits: `cancelled`, and `expired` when a reserved stall is not started within 30 minutes. A `queued` driver moves to `reserved` when a stall frees, and gets the `ev_reservation_ready` push.

Enum	Values
Reservation status	`pending_review`, `confirmed`, `reserved`, `queued`, `active`, `completed`, `expired`, `cancelled`
Stall status	`available`, `occupied`, `reserved`, `out_of_service`
Station status	`active`, `inactive`, `maintenance` (shown as "Under maintenance"; drivers cannot reserve)

Rules (Built (default) values, from settings):

1. One open reservation per driver. Queues hold up to 10 drivers.
2. The reservation fee is ₦500, taken from driver earnings when a stall is actually held. It may take earnings down to −₦5,000. Platinum and Diamond drivers pay nothing (`fee_waived`).
3. Cancelling before `free_cancellation_until` (2 minutes) refunds the fee. Show a countdown from that field. A no-show keeps the fee.
4. The fee shows in earnings history as `transaction_type` `ev_reservation_fee`.

What the platform does **not** do:

- **Live charging data.** There is no live battery percentage or kWh during a session. The driver can send `battery_level_percent` when going online. Not built.
- **Energy billing.** The platform charges only the ₦500 reservation fee. Stations show `pricing_per_kwh` (kobo) for information. Nothing records who pays for the energy. That is an open product decision. Not built.

The admin EV map can listen on the private channel `admin.ev-stations` for stall changes.

EV charge requests (fixed 22 October, PR #121)

A driver can also ask for a charging slot from the request form instead of reserving a stall straight away. Operations review each request. Before PR #121 nothing reviewed them.

Action	Endpoint	Who
Submit a request	`POST /driver/ev-reservations` with `battery_level`, optional `requested_date` (today or later), `requested_time`, `notes`, `station_id`	Driver with `can_charge`. Refused while another request or reservation is open
Review queue	`GET /admin/ev-charge-requests` (default: awaiting review and confirmed; filters `status` as a comma list, `station_id`, `city_id`, `search` by driver name or phone; `summary.pending_review`, `summary.confirmed`)	Operations
Confirm	`POST /admin/ev-charge-requests/{id}/confirm` with `confirmed_slot_at` (ISO 8601, from now up to 7 days ahead) and optional `station_id`	Operations
Decline	`POST /admin/ev-charge-requests/{id}/decline` with `reason` (5–500 characters, shown to the driver)	Operations

Lifecycle: `pending_review` → `confirmed` → at the slot, `reserved` (a stall is held, fee taken, check in within 30 minutes) or `queued` (station full; first in line). Exits: `cancelled` (declined, or cancelled by the driver), `expired` (not reviewed by the end of the requested day, or the slot was missed).

What the driver app shows on a confirmed request: `confirmed_slot` ("10:30 am"), `check_in_time` ("11:00 am", the latest arrival), `confirmed_slot_at`, `approver_name` (always "E-tiGo Operations"; staff names are never sent to drivers) and, when declined, `decline_reason`. Admins also get `reviewed_by`.

Rules (Built (default), not client-confirmed): slots up to 7 days ahead; the hold starts at the slot and lasts 30 minutes like any reservation; the ₦500 fee is taken when the stall is held, not when the request is confirmed; a driver whose earnings cannot cover the fee at the slot is cancelled and told.

Notifications: `ev_charge_request_confirmed`, `ev_charge_request_declined` (also sent when an unreviewed request expires), then the usual `ev_reservation_ready` or `ev_reservation_cancelled` at the slot.

Vehicle maintenance

Every car is tracked for routine service (5,000 km or 3 months), tyres and brakes (6 months) and its papers: insurance, roadworthiness certificate and vehicle licence. Plug-in cars add a battery health check (12 months). Overdue service only warns. **A paper expired for more than 7 days takes the car off the road**: the driver cannot go online until a renewed paper is approved. A trip in progress is never ended. All intervals are Built (default) Operations settings.

Action	Endpoint	Who
Vehicle status	`GET /driver/vehicle/status`	Driver
Upload a record	`POST /driver/vehicle/maintenance-records` (multipart; `proof` required, JPG, PNG or PDF up to 10 MB)	Driver, own car only. Fleet drivers get 403
Record history	`GET /driver/vehicle/maintenance-records`	Driver
Odometer	`POST /driver/vehicle/odometer`	Driver. A lower reading than the last returns 422
Repair requests	`GET/POST /driver/repairs`, `GET /driver/repairs/{id}`, `POST /driver/repairs/{id}/attachments`, `DELETE /driver/repairs/{id}/attachments/{attachmentId}`	Driver. Attachments: JPG, PNG, WEBP or PDF up to 10 MB
Fleet overview	`GET /admin/vehicle-maintenance` (filters `status`, `type`, `fleet_only`, `blocked_only`, `search`)	Operations
One car	`GET /admin/vehicles/{id}/maintenance`	Operations

Action	Endpoint	Who
Review queue	`GET /admin/vehicle-maintenance/records`, `POST /admin/vehicle-maintenance/records/{id}/review` (`approve`, or `reject` with a reason)	Operations
Record work, upload papers	`POST /admin/vehicles/{id}/maintenance-records`, `POST /admin/vehicles/{id}/documents`	Operations
Unlinked papers	`GET /admin/vehicle-documents/unlinked`, `POST /admin/vehicle-documents/{id}/link`	Operations
Settings	`GET/PUT /admin/settings/vehicle-maintenance`	Operations

Enum	Values
Maintenance item type	`service`, `tyres_brakes`, `insurance`, `roadworthiness`, `vehicle_licence`, `battery_health`
Item status	`ok`, `due_soon`, `overdue`, `expired`, `unknown` ("No record yet")
Record status	`pending`, `approved`, `rejected`
Repair request status	`in_progress`, `scheduled`, `completed`, `cancelled`; type `repair` or `maintenance`

Maintenance has **no cost fields**. Who pays for repairs is not recorded. Notifications: `vehicle_maintenance_due`, `vehicle_blocked` (ignores preferences), `vehicle_unblocked`, `maintenance_record_reviewed`.

Fleet agreement pause and swap

Action	Endpoint	Who
Pause, resume	`POST /admin/fleet-agreements/{id}/pause`, `/resume`	Operations, Finance
Swap vehicle	`POST /admin/fleet-agreements/{id}/swap` with `new_vehicle_id`, `reason`, `carry_over_remitted`	Operations, Finance

Action	Endpoint	Who
Excuse a day	`POST /admin/fleet-agreements/{id}/remittances/{remittance}/excuse`	Operations, Finance
End with settlement	`POST /admin/fleet-agreements/{id}/terminate-settle`	Operations, Finance

- Pause is **manual**. A maintenance record or a blocked car never pauses an agreement (Built).
- While paused, no remittance is taken and no daily target is created, so the shortfall streak neither grows nor resets (Built).
- **Correction:** vehicle swap **is built**. The draft said it was deferred. A swap terminates the old agreement and creates a new one. Add the Swap action to the agreement page.
- **Fixed (PR #121):** `GET /driver/fleet-agreement` now returns a paused agreement with `status: paused` and `paused_at`. `GET /driver/remittance/today` returns `agreement_status: paused`, `remittance: null` and "Your fleet agreement is paused. No remittance is due." Show an "Agreement paused" state. The driver also gets `fleet_agreement_paused` and `fleet_agreement_resumed` notifications, and a second agreement cannot be created while one is paused.

4. Third-party booking

Third-party booking is **Built** (PR #104), and the booker can choose who gets the PIN (PR #120). The draft said there was no evidence for it. The person who books is the **booker**. They stay the ride's passenger, pay, and own the ride. The person who travels is the **rider**, and needs no app. Guides: `FRONTEND_UPDATE_THIRD_PARTY_BOOKINGS.md` and `MOBILE_UPDATE_THIRD_PARTY_SHARE_WITH.md`.

The defaults below are Built (default) values, from the OQ-18 recommendation. The client has not confirmed them. Read every rule from `GET /config` → `third_party_bookings` and the ride's `third_party` object. Do not hard-code them.

Booking

Use `POST /rides` (now or with `scheduled_pickup_at`) and add:

Field	Rule
`rider_name`	Required, up to 100 characters

Field	Rule
`rider_phone`	E.164, or a Nigerian local number. The booker's own number makes it a normal ride
`rider_age_confirmed`	Must be `true`: "The rider is 18 or over, or an adult is travelling with them"
`share_with`	Optional, `rider` or `both`. Who gets the **PIN**. Build the radio buttons from `share_with_options` and pre-select `default_share_with`. Hide the question when the options list is empty

The response has `ride.is_third_party: true` and a `ride.third_party` object. When the PIN goes to the rider only, `pin_code` is `null`.

The draft's questions, answered

Area	Answer	Tag
Exists?	Yes, on staging	Built
Rider identity	Name and phone. The rider needs no account	Built
Payer	Always the booker. Wallet or card only; cash and bank transfer are off	Built (default)
Rider notification	WhatsApp first, SMS fallback, through Sendchamp. Sent **when a driver is assigned**, not at booking. The message has driver details, the PIN (if theirs) and a tracking link	Built
Pickup security	Yes. The rider reads the normal ride PIN to the driver	Built
Driver view	The new `rider` object: `first_name` and the real `phone` (not masked), which turns `null` after the trip. `passenger` (the booker) has only `id` and `first_name`	Built
Cancel, rate, tip, dispute	The booker only. The rider's message says to contact the booker for changes	Built
Receipt	The booker	Built
Scheduled for someone else	Yes. The rider gets a reminder before pickup without the PIN, then the PIN at dispatch	Built

Area	Answer	Tag
Limits	2 active third-party rides per booker; 10 different rider numbers per 24 hours	Built (default)
Tracking	Both rider and booker by default (`tracking_recipient`). Not part of the booker's choice	Built (default)

What the booker's app shows

- Drive the PIN and map from `ride.third_party.booker_sees_pin` and `booker_sees_tracking`. If the booker is not allowed tracking, `GET /rides/{ride}/location` returns 403.
- Word the PIN area from `rider_notification_status`: `pending`, `sent_whatsapp`, `sent_sms` or `failed` (`delivered` is reserved, not sent yet).
- If the rider can't be reached, the status becomes `failed`, `booker_sees_pin` turns true, and the booker gets the `third_party_rider_unreachable` push. Show the PIN with "Please share this PIN with Amina".
- `GET /passenger/active-ride` returns the booker's own ride in `ride`, and rides booked for others in `third_party_rides`. `GET /rides?is_third_party=1` lists only rides for others.
- Live updates: `.ride.rider_notification.updated` on `private-user.{booker_id}`. The PIN is never broadcast. Fetch `GET /rides/{ride}` when `booker_sees_pin` turns true.

Rider tracking page

The rider's link opens `GET /track/{ride}/{token}` on the API host: status, driver photo, car and plate, ETA, map, a Call 112 button and the safety line. It needs no login and expires 1 hour after the trip.

Safety

SOS works for third-party rides. The incident carries `telemetry_data.third_party_rider` and `triggered_by_booker`. When the booker raises it, the position is the car's.

Admin

`GET/PUT /admin/settings/third-party` (Operations): `enabled`, `pin_recipient`, `tracking_recipient`, `allow_cash`, `allow_bank_transfer`, `max_active_per_booker`, `max_rider_numbers_per_day`, `rider_trip_updates`. Changes apply to new bookings only. Ride detail should show "Booked by Tunde for Ada" and the delivery status.

The draft's privacy proposals mostly match what was built. One difference: the driver sees the rider's real phone number, not a masked one.

5. Disputes, offline flags and Lost & Found

All three case flows are **Built**. Disputes follow a three-tier ladder plus a separate Safety queue. Limits and targets are Built (default) values from the OQ-21 recommendation, not yet client-confirmed. Super admins can change them in `/admin/settings/disputes`. Guide: `FRONTEND_UPDATE_ADVANCED_DISPUTES.md`.

Who handles a dispute

Stage	Who	Refund they can approve alone	Response target	Resolution target
Tier 1	Support	Up to ₦5,000	24 hours	72 hours
Tier 2	Support Lead	Up to ₦50,000	24 hours	120 hours
Tier 3	Super Admin	No limit		
Safety queue	Safety Operator	Up to ₦50,000	60 minutes	24 hours

- A `safety_concern` case goes straight to the Safety queue. Support roles get 403 on safety cases. Safety Operators can read every case.
- A tier 1 case that misses a target moves to tier 2 on its own (`escalation.source: system`). Missed safety and tier 2 targets email the super admins.
- **Above ₦50,000** (the draft's open question): the refund waits as a refund request. Finance or a Super Admin approves it. Every **card** refund needs Finance (or Super Admin) approval, whatever the amount.
- Nobody can approve their own refund request, or a refund on a case where they made the original decision.

Endpoints: apps (passenger and driver)

Action	Endpoint	Notes
File	`POST /rides/{ride}/dispute`	Completed rides, or a charged late-cancellation fee. One dispute per person per ride. No time limit after the trip
My disputes	`GET /disputes`, `GET /disputes/{id}`	`/passenger/disputes` still works for older apps
Add evidence	`POST /disputes/{id}/evidence`	`files[]`, 1–5 per upload, up to 10 per case. Photos and PDFs up to 5 MB, audio up to 10 MB
Appeal	`POST /disputes/{id}/appeal` with `reason` (10–2000 characters)	Once, within 7 days of the decision

Show `stage_label` as the status line: "With our support team", "With our senior team", "With our safety team" or "Decision made". Show **Add evidence** while `can_add_evidence` is true, and **Appeal** while `appeal.can_appeal` is true. Admin names are never sent.

Endpoints: admin

Area	Endpoints	Roles
Queue and detail	`GET /admin/disputes` (filters `queue`, `tier`, `assigned`, `sla`, `appealed`, `pending_refund`, `open`, `status`, `search`, `sort=due`; `summary` for tab counts), `GET /admin/disputes/{id}`	Support, Support Lead, Safety
Actions	`POST /{id}/assign`, `DELETE /{id}/assign`, `PATCH /{id}/status`, `POST /{id}/escalate` (with `to_safety` for Safety), `POST /{id}/deescalate`, `POST /{id}/resolve`	Show only what `available_actions` lists
Internal notes	`GET/POST /admin/disputes/{id}/notes`, `PUT/DELETE /{id}/notes/{note}`	Never shown to the reporter

Area	Endpoints	Roles
Refund requests	`GET /admin/dispute-refund-requests`, `POST /{id}/approve`, `POST /{id}/reject`	Support, Support Lead, Safety, Finance. Use `can_approve` and `can_withdraw`
Settings	`GET/PUT /admin/settings/disputes`	Super Admin

Resolving takes `refund_method` (`wallet` by default, or `card`) and `clawback_driver_earnings` (default true). A 202 means the refund waits for approval and the case stays open.

Enums (the draft's proposed lifecycle does not exist)

Enum	Values
Status	`open`, `under_review`, `resolved`, `dismissed`. An appeal reopens a resolved or dismissed case
Category	`fare_dispute`, `driver_behaviour`, `route_deviation`, `vehicle_condition`, `safety_concern`, `payment_issue`, `item_left_behind` (use Lost & Found instead), `other`
Queue	`support`, `safety`
Appeal outcome	`pending`, `upheld`, `overturned`
SLA status	`on_track`, `at_risk`, `breached`; `met`, `missed` once closed; `none`
Evidence type	`photo`, `audio`, `document`
Refund method	`wallet`, `card`
Refund request status	`pending`, `approved`, `rejected`, `cancelled`

The draft's `awaiting_evidence`, `escalated` and `closed` statuses do not exist. Escalation shows as a tier change and a new `stage_label`.

Entry points. A dispute can be raised only from a ride (a completed trip or a charged late-cancellation fee). There is no dispute entry for a payment row, fleet remittance, driver conduct outside a trip, or maintenance (Not built).

Fleet fares (the draft's cross-module question). A refund on a fleet driver's ride claws back only the driver's share of that fare. E-tiGo bears its own remittance share. Remittance records (`remitted_amount`, `target_met`) are not reopened (Built).

Notification: `dispute_update` with `data.dispute_id` and `data.stage`.

Offline trip flags

Compliance flags on drivers who may finish trips off the app. Admin: Operations and Safety. Guides: `MOBILE_UPDATE_OFFLINE_FLAGS.md`, `FRONTEND_UPDATE_OFFLINE_FLAGS_DESIGN_GAPS.md`.

Concept	Rule (Built)
Levels and sanctions	L1 warning, L2 48-hour suspension, L3 pending deactivation. The sanction starts when the flag is raised, not when an admin upholds it
Statuses	`flagged`, `disputed`, `upheld`, `overturned`. An upheld flag can still be overturned on appeal. An overturned flag is final
Driver response	The driver disputes once, before review: `POST /driver/flags/{flag}/dispute`. There is **no driver appeal endpoint**: a driver appeals through support. Correction to the draft
L3 response window	48 hours from reaching L3 (`deactivation_response_due_at`). Until then, unless the driver has disputed, Uphold and Confirm deactivation return 422 for Operations and Safety. A super admin can confirm early
Verified cash-taking	`POST /admin/offline-flags/{id}/verify-cash-taking` moves a flag to L3
Deactivation	`POST /admin/offline-flags/{id}/confirm-deactivation` requires `notes` (10–2000). Upholds an unreviewed flag in the same call
Undo a deactivation	Super admin overturn only. De-escalate is refused

Admin endpoints: `GET /admin/offline-flags`, `GET /{id}`, `POST /{id}/review`, `/escalate`, `/de-escalate`, `/confirm-deactivation`, `/verify-cash-taking`. Use `flag.allowed_actions` for the buttons. Driver: `GET /driver/compliance`. Push `data.type`: `compliance_warning`.

Lost & Found

Guide: `FRONTEND_UPDATE_LOST_AND_FOUND.md`. A passenger reports an item, the driver answers, the two arrange the return in the app, and support steps in when needed. Drivers can also hand in items they find.

Action	Endpoint	Who
Report or hand in	`POST /rides/{ride}/lost-items` (`category`, `description` 5–1000, optional `contact_phone`, up to 3 `photos[]`)	Passenger or driver, within 30 days of the trip
List, detail	`GET /lost-items` (`ride_id`, `open=1`), `GET /lost-items/{id}`	Both
Driver answer	`POST /lost-items/{id}/respond` `{ found, note? }`, `POST /{id}/returned`	Driver
Passenger	`POST /{id}/received`, `/escalate`, `/cancel`	Passenger
Messages	`POST /lost-items/{id}/messages`	Both
Queue	`GET /admin/lost-items`, `GET /{id}`, `POST /{id}/intervene` (`note`, `message`, `mark_found`, `resolve`, `close`)	Support, Support Lead, Operations
Settings	`GET/PUT /admin/settings/lost-and-found`	Operations

Statuses: `reported`, `driver_notified`, `driver_confirmed`, `driver_denied`, `returned`, `resolved`, `unresolved`, `cancelled`. **Closure reasons:** `not_found`, `unclaimed`, `passenger_unreachable`, `driver_unreachable`, `other`. **Categories:** `phone`, `wallet_or_purse`, `bag`, `keys`, `id_or_documents`, `electronics`, `clothing`, `glasses`, `jewellery`, `other`.

Handover: once the driver says "I have it", both see each other's first name and phone (`share_contact_details`, Built (default)). The driver marks it returned and the passenger confirms. It auto-resolves after 72 hours without confirmation. A driver who doesn't answer within 24 hours, or a return later than 7 days, sends the report to support. The return fee is ₦0 (OQ-36, client not decided).

Notifications: `lost_item_report` (driver, needs an answer), `lost_item_update`, `lost_item_message`.

Cross-module dependencies

Every dependency the draft listed now has a rule in the code. Two needed backend work and were fixed on 22 October in PR #121: the driver's view of a paused agreement, and Operations review of EV charge requests.

From	To	Rule	Tag
Dispute	Finance	Refunds go to the wallet by default as a `refund` row. Above the decider's limit, or any card refund, they wait for approval	Built (default)
Dispute	Fleet remittance	A refund claws back only the driver's share of that fare. Remittance records are not reopened	Built
Maintenance	Fleet agreement	Pause is manual. Expired papers block driving but never pause an agreement	Built
Fleet pause	Shortfall streak	No daily target is created while paused, so the streak is frozen	Built
Fleet pause	Driver app	`GET /driver/fleet-agreement` returns the paused agreement; today's remittance says none is due	Built (PR #121)
Driver shifts	Fleet agreement	Shift mode is refused for hire-purchase cars, so shifts never touch a daily target	Built
EV charging	Finance	Only the ₦500 reservation fee touches the ledger (`ev_reservation_fee`). No energy billing	Built; energy billing Not built
Scheduled rides	Third-party booking	A scheduled ride can be booked for someone else	Built

From	To	Rule	Tag
Third-party booking	Disputes and Finance	The booker pays, gets the receipt, and is the only one who can dispute, tip or rate	Built
Scheduled rides	Finance	An unpaid late-cancellation fee becomes a balance due, which blocks new bookings	Client decision
Lost & Found	Finance	A return fee, if ever switched on, is charged on confirmation or becomes a balance due. It is ₦0 today	Built (default)

Answers to the 21 backend questions

The numbers match the draft's consolidated list. Details are in each module above.

#	Question	Answer	Tag
1	Balance types and field names	Passenger wallet: `balance`, `available`, `held`. Driver earnings: `pending`, `available`, `total_paid`. All in kobo	Built
2	Transaction types and statuses; fleet remittance type?	Direction `debit` or `credit`, plus `transaction_type` with 14 values including its own `fleet_remittance`. Top-ups: `pending`, `completed`, `abandoned`, `failed`	Built
3	Payout statuses, minimum, bank verification	`requested`, `approved`, `processing`, `paid`, `failed`, `reversed`, `rejected`. Minimum ₦1,000. Bank account: resolve, name check, OTP, then a 24-hour hold on a replacement account	Built (default)
4	Regular-driver commission on the ledger	20% by default, set by Finance. Shown as a separate `commission` debit beside the full `ride_fare` credit	Built (default)

#	Question	Answer	Tag
5	Export formats	CSV only. No PDF	Built
6	Do rider-scheduled rides exist?	Yes. 30 min to 3 days ahead, 3 per passenger, edit until 15 min before, late-cancellation fee within 30 min, quote locked to the pickup time	Client decision (window, fees); the rest Built (default)
7	Shift and plan statuses; driver shifts screen?	Shifts: `scheduled`, `active`, `completed`, `missed`, `cancelled`. Plans have no status; they are optional. Yes: `GET /driver/shifts`, shown when `shift_driver` is true	Built
8	Does shift mode change the fleet target or streak?	No. Shift mode is refused for hire-purchase cars	Built
9	Charging session lifecycle, live data, billing, payer	`reserved` → `active` → `completed`, or `expired`, `cancelled`. No live kWh or percentage. Only the ₦500 reservation fee is billed. Who pays for energy is undecided	Built; billing Not built
10	Maintenance statuses, categories, who submits, photo limits	Items: `service`, `tyres_brakes`, `insurance`, `roadworthiness`, `vehicle_licence`, `battery_health`. Records: `pending`, `approved`, `rejected`. Own-car drivers submit; Operations record for fleet cars. One proof file, JPG, PNG or PDF, 10 MB	Built
11	Auto-pause on maintenance; streak during a pause	No auto-pause. The streak is frozen while paused	Built
12	Is hybrid eligible for EV screens?	No. Only `electric` and `plug_in_hybrid` (`can_charge`)	Built

#	Question	Answer	Tag
13	Third-party booking: built?	Yes. See section 4 for the full question table	Built (default)
14	Dispute statuses, categories, entry points, outcomes	`open`, `under_review`, `resolved`, `dismissed`. 8 categories. Entry from a completed ride or a charged late-cancellation fee only. Appeal outcomes `upheld`, `overturned`	Built
15	Evidence rules and SLA timers	1–5 files per upload, 10 per case. Photos and PDFs 5 MB, audio 10 MB. Tier 1: 24 h response, 72 h resolution. Tier 2: 24 h, 120 h. Safety: 60 min, 24 h	Built (default)
16	Who approves refunds above ₦50,000?	Finance or a Super Admin. Card refunds always need Finance	Built (default)
17	Fleet-fare reversal rule	The driver loses only their share of the refunded fare. Remittance is not reopened	Built
18	Lost & Found statuses and handover	8 statuses (section 5). Driver confirms, contacts shared, driver marks returned, passenger confirms or it auto-resolves after 72 h. Return fee ₦0	Built (default); fee: OQ-36 open
19	Time zone for date fields	Every timestamp is UTC, ISO 8601. Send `scheduled_pickup_at` with an offset. Display in local time (Lagos). Business days such as fleet settlement use Africa/Lagos	Built

#	Question	Answer	Tag
20	Websocket or polling?	Websocket for rides (`ride.{id}` : status, driver location, payment), driver offers (`driver.{id}`), third-party delivery status (`user.{id}`), and admin `admin.rides` , `admin.sos` , `admin.ev-stations` . Poll for wallets, payouts, disputes, Lost & Found, maintenance and shifts, and refresh on each push	Built
21	Push, SMS and email triggers	In-app and push: every notification type listed per module. WhatsApp, then SMS: OTPs, third-party rider messages, bank-account change alerts. Email: admin alerts (missed shifts, maintenance digest, safety cases, missed dispute targets) and dispute decisions	Built

Testing

Use staging with the Postman collection *E-tiGo API v1* and the *E-tiGo — Staging* environment. Every OTP on staging is `123456` . Every seeded password is `Qwer!234` .

Seeded accounts

Who	Login	Use for
Super Admin	`admin@etigo.com`	Tier 3 disputes, dispute settings, early L3 confirm, inviting a Support Lead
Operations	`ops@etigo.com`	Shifts, maintenance, EV stations, scheduled-ride assignment, offline flags, Lost & Found
Safety Operator	`safety@etigo.com`	Safety dispute queue, offline flags, SOS

Who	Login	Use for
Support	`support@etigo.com`	Tier 1 disputes, read-only wallet check
Finance	`finance@etigo.com`	Wallets, payouts, adjustments, refunds, card-refund approval
Passenger	`+2348100000001` (Ade) and four more up to `…005`	Wallet, scheduled rides, third-party booking, disputes, Lost & Found
Driver, petrol	`+2348200000001` (Bayo, Toyota Corolla, online)	Payouts, scheduled-ride reservations
Driver, hybrid	`+2348200000002` (Kemi, Honda Accord, offline)	EV screens must stay hidden (`can_charge: false`)
Driver, electric	`+2348200000003` (Segun, Mercedes-Benz EQE, online)	EV stall reservations
Driver, CNG	`+2348200000005` (Tunde, Nissan Sentra, **suspended**)	Suspended state

Corrections to the draft's test data:

- **No Support Lead account is seeded.** Invite one as Super Admin with role `support_lead`.
- **No fleet agreement is seeded.** `+2348200000001` is a regular driver, not a fleet driver. To test fleet flows, Operations or Finance creates a fleet vehicle (`POST /admin/fleet-vehicles`, with `fuel_type`), assigns a driver and creates an agreement (`POST /admin/fleet-agreements`).
- The draft's quick flow "pause an agreement, then check `GET /driver/fleet-agreement` shows `paused`" works once PR #121 is on staging.

Demo data on staging

- `EvStationsDemoSeeder`: EV stations with stalls for the station screens.
- `OfflineFlagsDemoSeeder`: 22 flags for 11 Lagos drivers, every status at every level. Find `flag_id` with `GET /admin/offline-flags?search=`. Useful cases: Musa Garba (L3 past 48 hours, Confirm works), Funke Ajayi (L3 inside 48 hours, Uphold and Confirm return 422), Chidi Nwosu (L3 disputed), Segun Alabi (deactivation confirmed, super admin only), Yemi Ogunleye (L2 overturned on appeal).

- Rider messages need a funded Sendchamp wallet (it was empty on 19 October). When a message can't be delivered, `rider_notification_status` turns `failed` after about a minute and the booker sees the PIN. That is the "rider not reached" flow.

Quick flows

1. **Finance:** top up as Ade → list wallets as Finance → read the ledger entry → create an adjustment as one Finance admin and approve it as another → request a payout as Bayo → approve it as Finance → open the same wallet as Support and check there are no actions.
2. **Scheduled ride:** book as Ade 1 hour ahead → reserve it as Bayo → cancel as Ade inside 30 minutes → see the fee, or the balance due if it could not be charged.
3. **Third-party:** book as Ade with `rider_phone` and `share_with: rider` → confirm `pin_code` is `null` → wait for `failed` → confirm `booker_sees_pin` turns true.
4. **EV:** list stations as Ops → reserve a stall as Segun → cancel within 2 minutes and check the fee refund → try the same as Kemi and expect 422.
5. **Dispute:** file on a completed ride as Ade → take it as Support → resolve with a wallet refund above ₦5,000 → approve the refund request as Finance → see the `refund` row in Ade's wallet.
6. **Maintenance:** upload a service record as Bayo → approve it as Ops in the records queue → check `GET /driver/vehicle/status`.